ASSIGNMENT 2

Student Course Registration System

NAME: Ishmal Sheikh & Aamna Basharat

ROLL NO: 24P-3054 & 24P-3121

SECTION: BSE-4B

COURSE: SOFTWARE QUALITY ENGINEERING

Part A: Black-Box Testing

A1. Equivalence Class Partitioning

Input data is divided into valid and invalid classes, and one representative value is tested from each class.

Table A1: Equivalence Class Partitioning

Test ID	Requirement ID	Input Area	Equivalence Class	Class Type	Representative Input	Expected Result
ECP-01	FR-01, FR-02	Current credits	0–18 inclusive	Valid	10	Accepted for credit calculation
ECP-02	FR-02	Current credits	Below 0	Invalid	-1	ValueError: Invalid current credits
ECP-03	FR-02	Current credits	Above 18	Invalid	19	ValueError: Invalid current credits
ECP-04	FR-01, FR-02	Course credits	1–4 inclusive	Valid	3	Accepted for credit calculation
ECP-05	FR-02	Course credits	Below 1	Invalid	0	ValueError: Invalid course credits
ECP-06	FR-02	Course credits	Above 4	Invalid	5	ValueError: Invalid course credits
ECP-07	FR-04, FR-09	Outstanding dues	Exactly zero	Valid	Rs. 0	No dues block
ECP-08	FR-04, FR-09	Outstanding dues	Positive amount	Valid input, registration blocked	Rs. 1	DUES_BLOCK
ECP-09	FR-15	Outstanding dues	Negative amount	Invalid	Rs. -1	ValueError: Invalid dues
ECP-10	FR-03, FR-06	Prerequisites	No prerequisites required	Valid	Empty prerequisite set	Prerequisites satisfied
ECP-11	FR-03, FR-06	Prerequisites	All prerequisites completed	Valid	All required courses completed	Prerequisites satisfied
ECP-12	FR-03, FR-06	Prerequisites	At least one missing	Invalid for registration	One required course missing	PREREQUISITES_MISSING
ECP-13	FR-08, FR-09	Availability	At least one seat available	Valid	Capacity 30, enrolled 29	REGISTERED if other checks pass
ECP-14	FR-08, FR-09	Availability	Course exactly full	Valid input, waitlisted	Capacity 30, enrolled 30	WAITLISTED if other checks pass
ECP-15	FR-15	Availability	Invalid capacity or enrollment	Invalid	Capacity 0, or enrollment below 0 or above capacity	ValueError: Invalid capacity or enrollment
ECP-16	FR-05, FR-09	Duplicate registration	Course already registered	Invalid for new registration	Course code already in registered courses	ALREADY_REGISTERED

A2. Boundary Value Analysis

Values at, just below and just above each limit are tested, since errors tend to occur at the edges of valid ranges.

Table A2: Boundary Value Analysis

Test ID	Requirement ID	Parameter	Boundary	Input	Expected Result
BVA-01	FR-02	Current credits	Just below minimum	-1	ValueError: Invalid current credits
BVA-02	FR-02	Current credits	Minimum	0	Accepted
BVA-03	FR-01	Current credits	Just above minimum	1	Accepted
BVA-04	FR-01	Combined credits	Exact maximum	Current 17 + course 1 = 18	True
BVA-05	FR-01	Combined credits	Just above maximum	Current 18 + course 1 = 19	False
BVA-06	FR-02	Current credits	Maximum	18	Accepted as current credits
BVA-07	FR-02	Current credits	Just above maximum	19	ValueError: Invalid current credits
BVA-08	FR-02	Course credits	Just below minimum	0	ValueError: Invalid course credits
BVA-09	FR-02	Course credits	Minimum	1	Accepted
BVA-10	FR-02	Course credits	Maximum	4	Accepted
BVA-11	FR-02	Course credits	Just above maximum	5	ValueError: Invalid course credits
BVA-12	FR-01	Combined credits	Exact maximum	Current 14 + course 4 = 18	True
BVA-13	FR-01	Combined credits	Just above maximum	Current 15 + course 4 = 19	False
BVA-14	FR-08, FR-15	Course capacity	One seat remaining	Capacity 1, enrolled 0	REGISTERED
BVA-15	FR-08	Course capacity	Exactly full	Capacity 1, enrolled 1	WAITLISTED
BVA-16	FR-15	Enrollment	Below minimum	Capacity 1, enrolled -1	ValueError: Invalid capacity or enrollment
BVA-17	FR-15	Enrollment	Above capacity	Capacity 1, enrolled 2	ValueError: Invalid capacity or enrollment
BVA-18	FR-15	Capacity	Below minimum	Capacity 0, enrolled 0	ValueError: Invalid capacity or enrollment
BVA-19	FR-04, FR-09	Outstanding dues	Zero boundary	Rs. 0	No dues block
BVA-20	FR-04, FR-09	Outstanding dues	Smallest positive amount	Rs. 1	DUES_BLOCK

A3. Decision Table Testing

The decision table lists combinations of conditions and the expected registration result. When more than one condition fails, the first check in this priority order decides the outcome: outstanding dues, duplicate registration, missing prerequisites, credit-hour limit, seat availability.

Table A3: Registration Decision Table

Rule	Dues > 0?	Already registered?	Prerequisites missing?	Credit limit exceeded?	Seat available?	Expected outcome
R1	Yes	Yes	Yes	Yes	No	DUES_BLOCK
R2	No	Yes	Yes	Yes	No	ALREADY_REGISTERED
R3	No	No	Yes	Yes	No	PREREQUISITES_MISSING
R4	No	No	No	Yes	No	CREDIT_LIMIT
R5	No	No	No	No	No	WAITLISTED
R6	No	No	No	No	Yes	REGISTERED
R7	Yes	No	No	No	Yes	DUES_BLOCK
R8	No	Yes	No	No	Yes	ALREADY_REGISTERED

Table A3.1: Meaning of the decision table rules

Rule	Explanation
R1	Dues block registration even when other conditions also fail.
R2	Duplicate registration is checked before prerequisites and credit eligibility.
R3	Missing prerequisites block registration before credit and seat checks.
R4	Exceeding the credit limit blocks registration before the seat check.
R5	An eligible student is waitlisted when the course is full.
R6	An eligible student is registered when a seat is available.
R7	Positive dues block registration even when all later checks pass.
R8	Duplicate registration blocks registration even when all later checks pass.

The table covers all six distinct outcomes: DUES_BLOCK, ALREADY_REGISTERED, PREREQUISITES_MISSING, CREDIT_LIMIT, WAITLISTED and REGISTERED. R1–R4, R7 and R8 also show the priority order.

A4. Pytest Implementation and Execution

Table A4.1: Initial Black-Box Test Execution Summary

Description	Actual Result
Test file	tests/test_blackbox.py
Testing framework	pytest
Total tests executed	36
Passed tests	26
Failed tests	10
Execution time	2.31 seconds
Result	10 failures identified in original implementation

The black-box suite was executed against the original implementation. Of 36 tests, 26 passed and 10 failed. The failures indicate possible defects in dues validation, prerequisite checking, credit-hour limits and course capacity handling.

Part B: White-Box Testing

B1. CFG and Cyclomatic Complexity

B1.1 Control Flow Graph

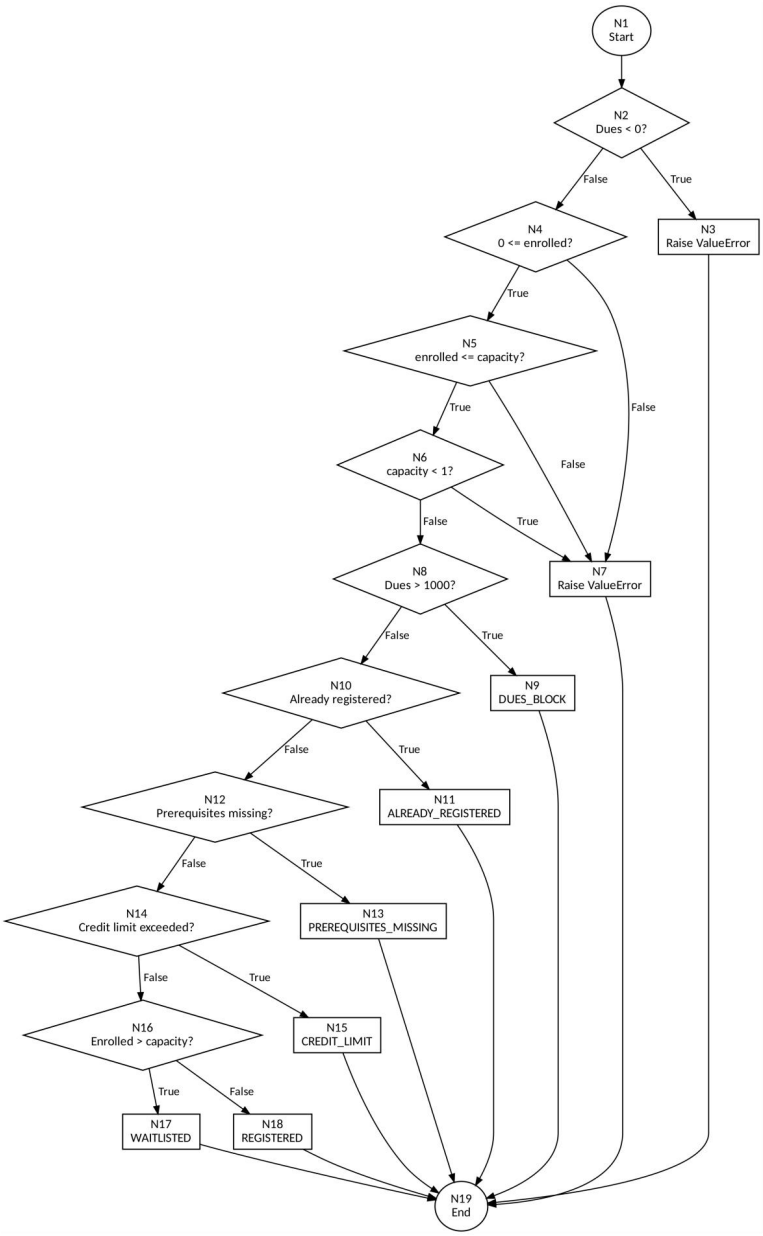


Figure B1.1: Control Flow Graph of registration_decision()

B1.2 CFG Nodes

Node	Type	Description
N1	Start	Start of registration decision
N2	Decision	Outstanding dues less than zero?
N3	Return	Raise invalid dues error
N4	Decision	Enrollment greater than or equal to zero?
N5	Decision	Enrollment less than or equal to capacity?
N6	Decision	Capacity less than one?
N7	Return	Raise invalid capacity error
N8	Decision	Outstanding dues greater than 1000?
N9	Return	DUES_BLOCK
N10	Decision	Course already registered?
N11	Return	ALREADY_REGISTERED
N12	Decision	Prerequisites missing?
N13	Return	PREREQUISITES_MISSING
N14	Decision	Credit limit exceeded?
N15	Return	CREDIT_LIMIT
N16	Decision	Enrollment greater than capacity?
N17	Return	WAITLISTED
N18	Return	REGISTERED
N19	End	End of function

B1.3 Cyclomatic Complexity

V(G) = E − N + 2P, where E is the number of edges, N the number of nodes and P the number of connected components.

Element	Value
Nodes (N)	19
Edges (E)	27
Connected components (P)	1

Calculation: V(G) = 27 − 19 + 2(1) = **10**

Cross-check: the graph has nine decision nodes (N2, N4, N5, N6, N8, N10, N12, N14, N16), so V(G) = 9 + 1 = **10**.

The expanded short-circuit convention is used: the three atomic conditions in the capacity-validation expression (N4, N5, N6) are counted as separate decisions. If that expression were counted as one source-level decision, V(G) would be 8.

B1.4 Independent Basis Paths

Table B1.4: Basis path set

Path	Execution condition	Expected result	Representative test
P1	Outstanding dues are negative	ValueError	WB01
P2	Dues valid, enrollment is negative	ValueError	WB-C02
P3	Enrollment is greater than capacity	ValueError	WB-C03
P4	Capacity is zero	ValueError	WB-C04
P5	Dues are greater than 1000	DUES_BLOCK	WB02
P6	Course is already registered	ALREADY_REGISTERED	WB03
P7	Required prerequisites are missing	PREREQUISITES_MISSING	WB04
P8	Credit limit is exceeded	CREDIT_LIMIT	WB05
P9	Enrollment is greater than capacity at the waitlist check	WAITLISTED (unreachable)	None possible
P10	All checks pass	REGISTERED	WB06

Important finding: P9 is unreachable in the original implementation. The earlier capacity validation rejects enrollment greater than capacity, so the waitlist condition can never be True. This is a structural defect to document, not a test to force to pass.

B2. Statement and Branch Coverage

B2.1 Initial and Final Coverage

Coverage was measured with pytest-cov (branch coverage enabled).

Table B2.1: Coverage before and after white-box additions

Measure	Initial (black-box)	Final (combined)
Total tests	36	46
Passed tests	26	36
Failed tests	10	10
Total statements	53	53
Covered statements	49	50
Missing statements	4	3
Statement coverage	92.45%	94.34%
Total branches	26	26
Covered branches	21	22
Missing branches	5	4
Partially covered branches	3	2
Branch coverage	80.77%	84.62%
Combined coverage	88.61%	91.14%

Calculations:

statement coverage = covered statements ÷ total statements (49/53 = 92.45%; 50/53 = 94.34%).
Branch coverage = covered branches ÷ total branches (21/26 = 80.77%; 22/26 = 84.62%).

Combined coverage = (covered statements + covered branches) ÷ (total statements + total branches) (70/79 = 88.61%; 72/79 = 91.14%).

B2.2 Uncovered Locations

The initial report showed uncovered lines 51, 59, 70 and 71 of registration.py. After the white-box tests, line 51 is covered and these remain:

Line	Code location	Reason
59	return "WAITLISTED" in registration_decision()	Unreachable: the earlier capacity validation rejects enrollment above capacity.
70	elif decision == "WAITLISTED" in register_course()	No test can reach this branch while WAITLISTED is unreachable.
71	course.waitlist.append(student.student_id)	The waitlist append is never executed.

Added structural tests: WB-C01 to WB-C04 and WB01 to WB06 (Table B4.1).

The ten white-box tests raised statement coverage by 1.89 percentage points and branch coverage by 3.85 points. The remaining gaps all belong to the waitlist functionality, which cannot be executed without correcting the implementation.

B3. Condition Coverage and MC/DC

B3.1 Compound Decision and Atomic Conditions

```
if not 0 <= course.enrolled <= course.capacity or course.capacity < 1:
    raise ValueError("Invalid capacity or enrollment")
```

This decision contains three atomic conditions:

Condition	Meaning
A	0 <= course.enrolled
B	course.enrolled <= course.capacity
C	course.capacity < 1

The overall decision is D = ¬(A ∧ B) ∨ C. If D is True the program raises ValueError; if D is False it continues with the registration checks.

B3.2 Truth Table with Short-Circuit Evaluation

Python skips conditions once the result is known. N/E means Not Evaluated.

Test ID	A	B	C	Decision D	Expected result
WB-C01	T	T	F	F	REGISTERED
WB-C02	F	N/E	N/E	T	ValueError
WB-C03	T	F	N/E	T	ValueError
WB-C04	T	T	T	T	ValueError

WB-C01: enrollment is within capacity and capacity is valid, so all three conditions are evaluated.

WB-C02: enrollment is negative. A is False, so B and C are skipped.

WB-C03: enrollment is greater than capacity. A is True and B is False, so C is skipped.

WB-C04: enrollment and capacity are both zero. A and B are True, so C is evaluated and is True.

B3.3 MC/DC Independent Pairs

MC/DC requires each atomic condition to independently change the decision outcome. In each pair only the listed condition changes; the other conditions keep the same logical values.

Condition	Test pair	Change	Other conditions held at	Decision change
A	WB-C01, WB-C02	A: T → F	B = T, C = F	F → T
B	WB-C01, WB-C03	B: T → F	A = T, C = F	F → T
C	WB-C01, WB-C04	C: F → T	A = T, B = T	F → T

In WB-C02 and WB-C03, Python does not evaluate B and C (or just C), but their logical values are unchanged from WB-C01, so the short-circuit does not affect the pair. All three conditions therefore have an MC/DC pair using four tests (n + 1 for n = 3). Each condition also takes both True and False across the tests, which gives full condition coverage.

Condition coverage and MC/DC were calculated manually, because pytest-cov measures only statement and branch coverage.

B4. White-Box Pytest Tests

Ten tests were added in tests/test_whitebox.py. Each one targets a specific node, condition or path of registration_decision().

Table B4.1: White-box tests and results

Test ID	Test purpose	Targeted branch, condition or path	Result
WB-C01	Valid capacity conditions	A = T, B = T, C = F; N4–N6 pass and execution continues	PASS
WB-C02	Negative enrollment	Condition A False at N4 → N7 (P2)	PASS
WB-C03	Enrollment above capacity	Condition B False at N5 → N7 (P3)	PASS
WB-C04	Zero capacity	Condition C True at N6 → N7 (P4)	PASS
WB01	Negative dues validation	N2 True → N3 (P1)	PASS
WB02	Dues-block decision	N8 True → N9 (P5)	PASS
WB03	Duplicate registration	N10 True → N11 (P6)	PASS
WB04	Missing prerequisite	N12 True → N13 (P7)	PASS
WB05	Credit limit exceeded	N14 True → N15 (P8)	PASS
WB06	Successful registration	All checks pass → N18 (P10)	PASS

Table B4.2: White-box execution summary

Metric	Result
Total white-box tests	10
Passed	10
Failed	0
Execution time	0.39 seconds